

Université Polytechnique de Sfax

Matière : SoA et Microservices

# Compte Rendu — TP4

## Création d'une API RESTful avec Express.js

|                               |                                                 |
|-------------------------------|-------------------------------------------------|
| <b>Matière</b>                | SoA et Microservices                            |
| <b>Enseignant</b>             | Dr. Salah Gontara                               |
| <b>Classe</b>                 | 4Info                                           |
| <b>Année universitaire</b>    | 2025/2026                                       |
| <b>Technologies utilisées</b> | Node.js, Express.js, SQLite3, Keycloak, Swagger |

## Introduction

Ce TP a pour objectif de créer une API RESTful complète en utilisant le framework Express.js avec Node.js et une base de données SQLite3. Nous avons implémenté les quatre opérations CRUD (Create, Read, Update, Delete) sur une ressource "/personnes", sécurisé l'API avec OAuth 2.0 via Keycloak, et documenté les routes avec Swagger/OpenAPI.

## Étape 1 : Initialisation du Projet

### Objectif

Mettre en place l'environnement de développement Node.js avec les dépendances nécessaires.

### Commandes exécutées

#### Terminal — Initialisation

```
npm init -y  
npm install express sqlite3
```

### Analyse

La commande "npm init -y" crée automatiquement le fichier package.json sans poser de questions. Ce fichier contient les métadonnées du projet et la liste des dépendances. La commande "npm install" télécharge Express.js (framework web) et SQLite3 (base de données légère sans serveur) dans le dossier node\_modules.

### Conclusion

L'environnement est correctement initialisé. Express.js fournit le routage HTTP et SQLite3 assure la persistance des données dans un fichier local, ce qui est idéal pour le développement.

## Étape 2 : Configuration de SQLite3

### Objectif

Créer la connexion à la base de données et initialiser la table "personnes" avec des données de test.

### Points clés du fichier database.js

Le fichier database.js établit la connexion à une base SQLite locale. La table "personnes" est créée avec une clé primaire auto-incrémentée (AUTOINCREMENT) et un champ "nom" obligatoire (NOT NULL). L'utilisation de module.exports partage la connexion entre les fichiers du projet. Les paramètres préparés (?) dans les requêtes SQL protègent contre les injections SQL.

### Conclusion

Ce fichier joue le rôle de couche d'accès aux données (Data Access Layer). Il centralise la gestion de la base de données et garantit que la table est créée si elle n'existe pas encore, évitant ainsi les erreurs au redémarrage.

## Étape 3 : Mise en Place de l'API REST

## Objectif

Implémenter les 5 routes CRUD conformes aux bonnes pratiques REST.

## Tableau des routes implémentées

| Méthode HTTP | Route          | Action      | Description                    |
|--------------|----------------|-------------|--------------------------------|
| GET          | /personnes     | Read (tous) | Récupérer toutes les personnes |
| GET          | /personnes/:id | Read (un)   | Récupérer une personne par ID  |
| POST         | /personnes     | Create      | Créer une nouvelle personne    |
| PUT          | /personnes/:id | Update      | Modifier une personne          |
| DELETE       | /personnes/:id | Delete      | Supprimer une personne         |

## Conclusion

Les 5 routes respectent l'architecture REST en utilisant les méthodes HTTP appropriées. Le middleware `express.json()` parse automatiquement les corps de requête JSON. Chaque route retourne une réponse JSON structurée avec un message de succès ou d'erreur.

## Étape 4 : Modification de la Structure de la Base de Données

### Objectif

Ajouter un champ "adresse" à la table "personnes" et mettre à jour les routes POST et PUT.

### Modifications effectuées

La table "personnes" a été recréée avec le champ "adresse TEXT" en plus. Le fichier `maBaseDeDonnees.sqlite` a été supprimé pour forcer la recréation de la table avec la nouvelle structure. Les routes POST et PUT ont été mises à jour pour accepter et traiter le champ adresse en utilisant la syntaxe de destructuring ES6 : `const { nom, adresse } = req.body`.

### Conclusion

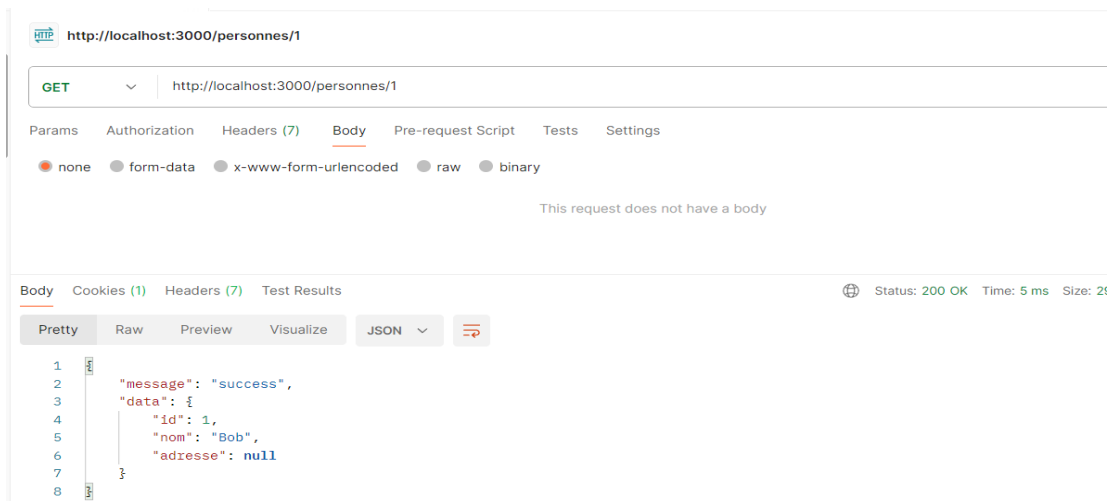
SQLite ne supporte pas facilement l'ajout de colonnes à une table existante dans ce contexte. La bonne pratique en développement est de supprimer le fichier `.sqlite` et de le laisser se recréer automatiquement. Il est important de maintenir la cohérence entre le schéma de la BDD et toutes les routes qui créent ou modifient des données.

## Étape 5 : Tests avec Postman

### Objectif

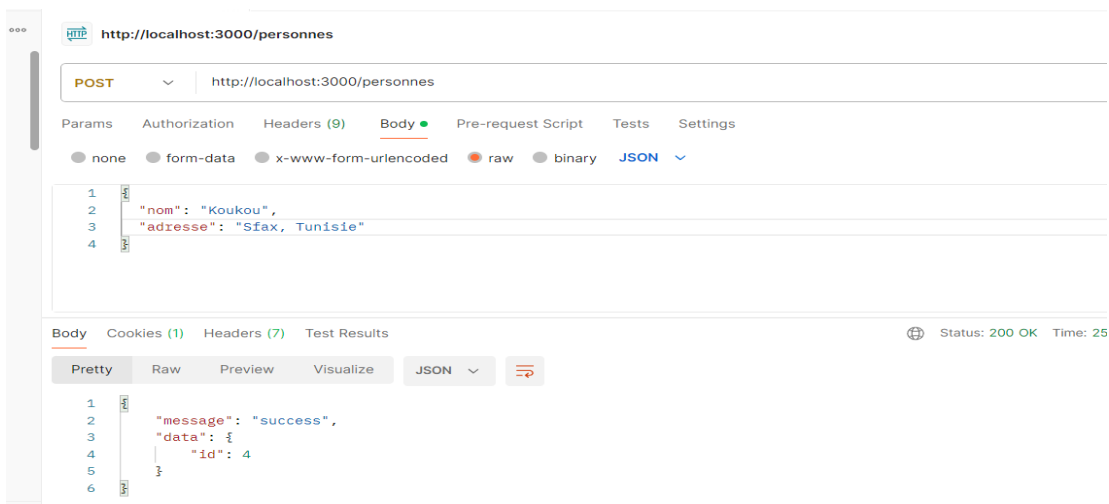
Valider le bon fonctionnement de toutes les routes en utilisant l'outil Postman.

### Test 1 — GET toutes les personnes



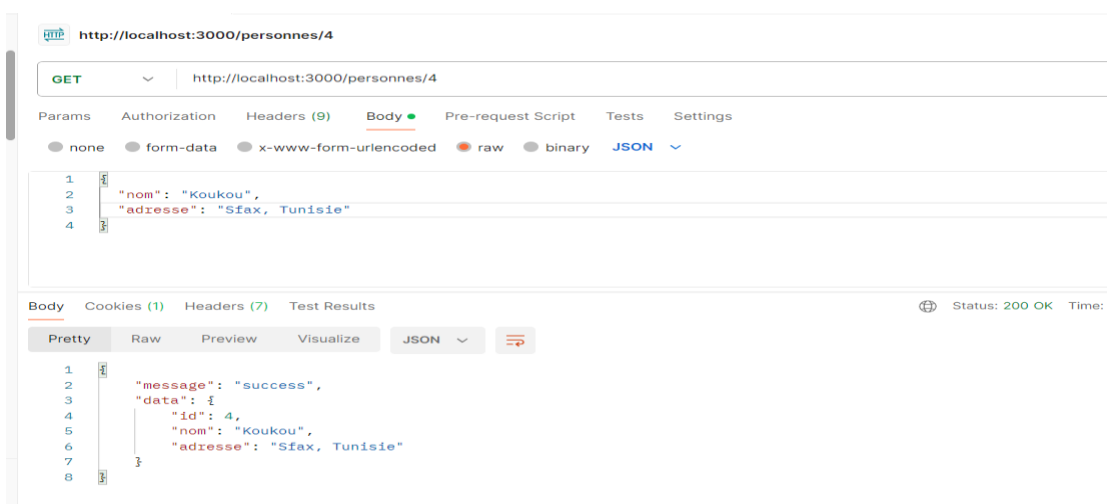
La requête GET /personnes retourne la liste de toutes les personnes stockées dans la base de données avec un statut 200 OK.

## Test 2 — GET une personne par ID



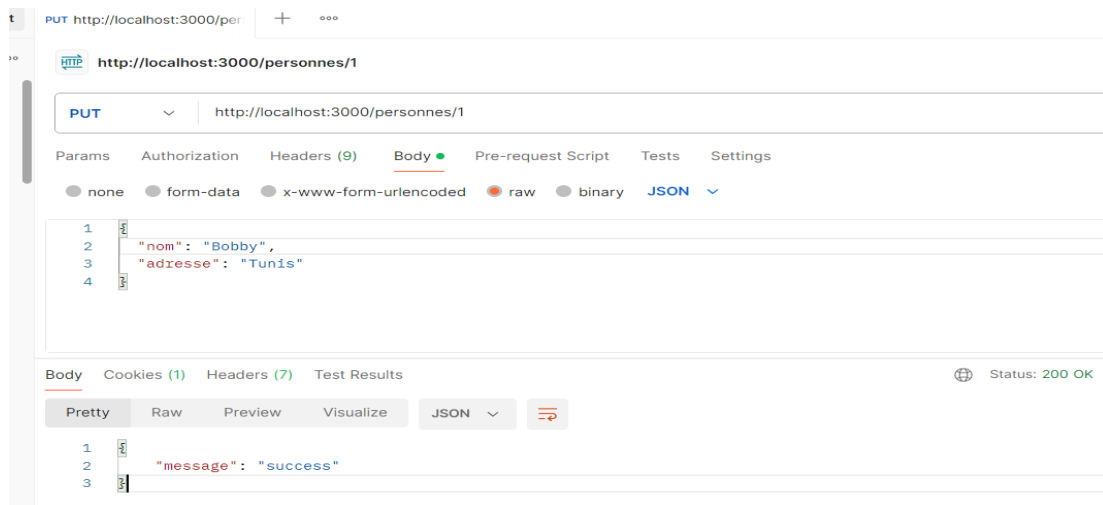
La requête GET /personnes/:id retourne les données de la personne correspondant à l'ID spécifié dans l'URL.

## Test 3 — POST créer une personne



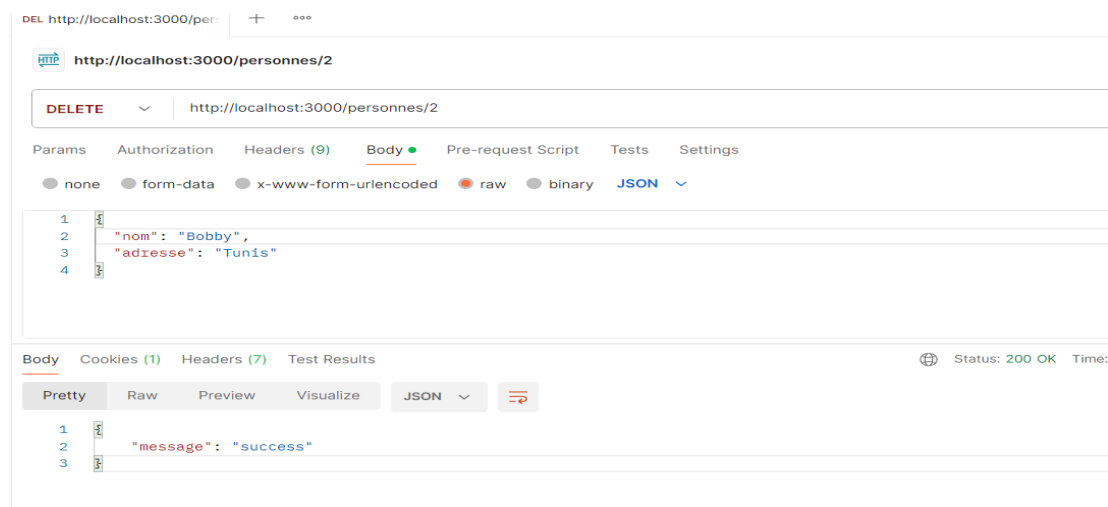
La requête POST /personnes avec un corps JSON contenant nom et adresse crée une nouvelle personne et retourne son ID généré automatiquement.

## Test 4 — PUT modifier une personne



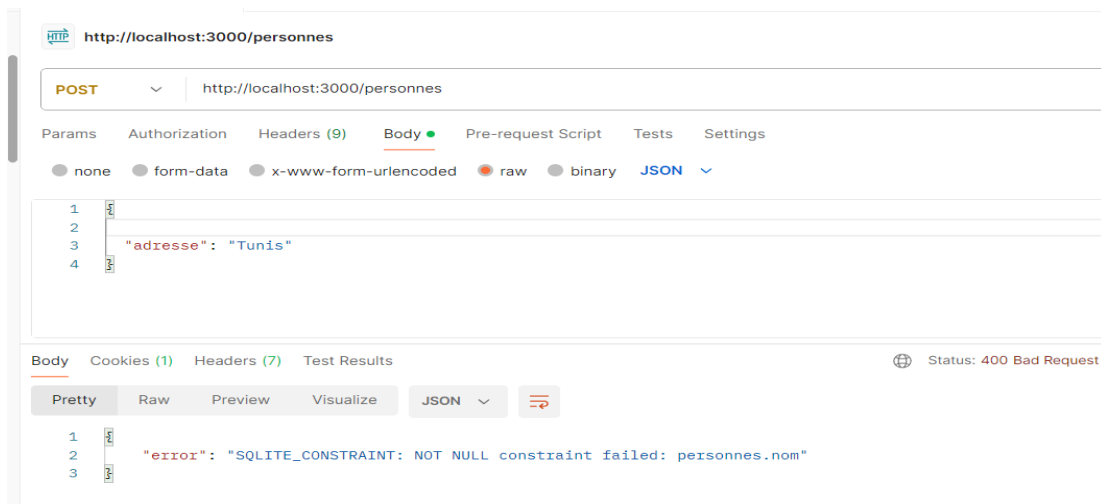
La requête PUT /personnes/:id met à jour les informations de la personne correspondant à l'ID. Un GET de vérification confirme la modification.

## Test 5 — DELETE supprimer une personne



La requête DELETE /personnes/:id supprime la personne correspondante et retourne un message de succès.

## Test 6 — Gestion des erreurs



Un POST avec des données incorrectes ou manquantes retourne un statut 400 Bad Request avec un message d'erreur explicite, validant la robustesse de la gestion des erreurs.

## Conclusion

Les tests Postman ont validé le bon fonctionnement des 5 routes CRUD. La gestion des erreurs fonctionne correctement en retournant des codes HTTP appropriés (200 pour le succès, 400 pour les erreurs client). La persistance des données a été vérifiée en effectuant des requêtes GET après chaque modification.

## Étape 6 (Optionnelle) : OAuth 2.0 avec Keycloak

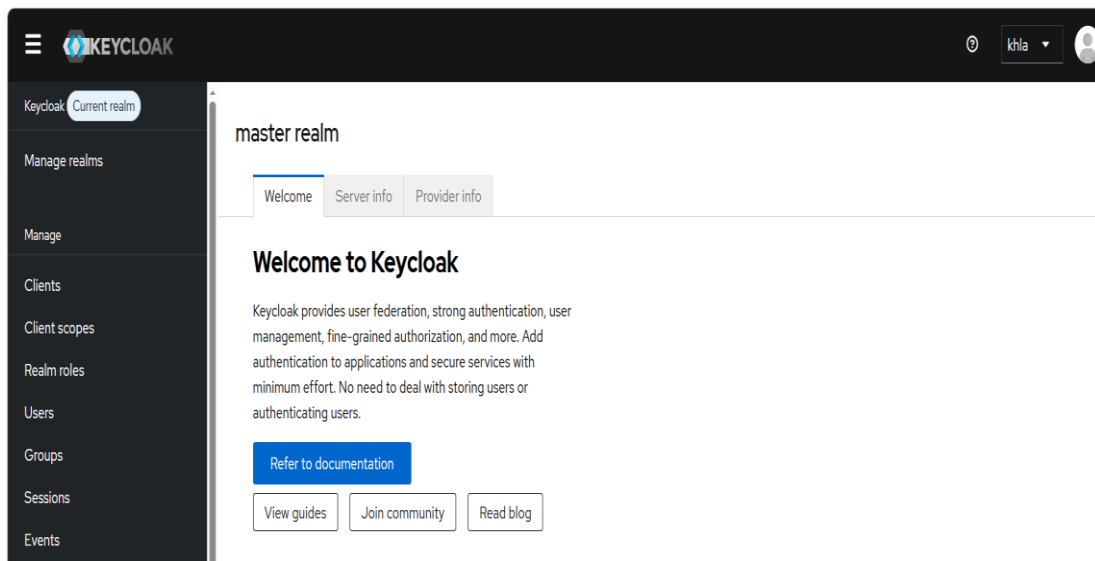
### Objectif

Sécuriser les routes de l'API avec le protocole OAuth 2.0 en utilisant Keycloak comme serveur d'identité.

### Concepts théoriques

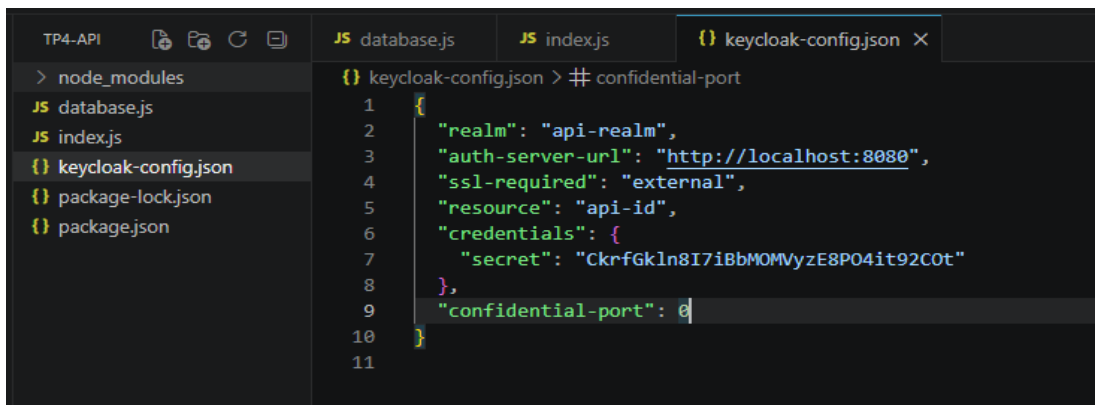
|                           |                                                                              |
|---------------------------|------------------------------------------------------------------------------|
| <b>OAuth 2.0</b>          | Protocole d'autorisation standard qui délivre des tokens d'accès temporaires |
| <b>Keycloak</b>           | Serveur d'identité open-source qui gère les utilisateurs, rôles et tokens    |
| <b>Realm</b>              | Espace isolé de configuration dans Keycloak (api-realm dans notre cas)       |
| <b>Client</b>             | Représente notre application API dans Keycloak (api-id)                      |
| <b>JWT</b>                | Token signé contenant l'identité et les droits de l'utilisateur              |
| <b>Bearer Token</b>       | Méthode d'envoi du token dans le header Authorization                        |
| <b>keycloak.protect()</b> | Middleware Express qui bloque l'accès sans token valide                      |

### Configuration Keycloak



Configuration du fichier keycloak-config.json avec les paramètres du realm api-realm et du client api-id. Ce fichier connecte notre API au serveur Keycloak et permet la vérification des tokens.

### Fichier keycloak-config.json

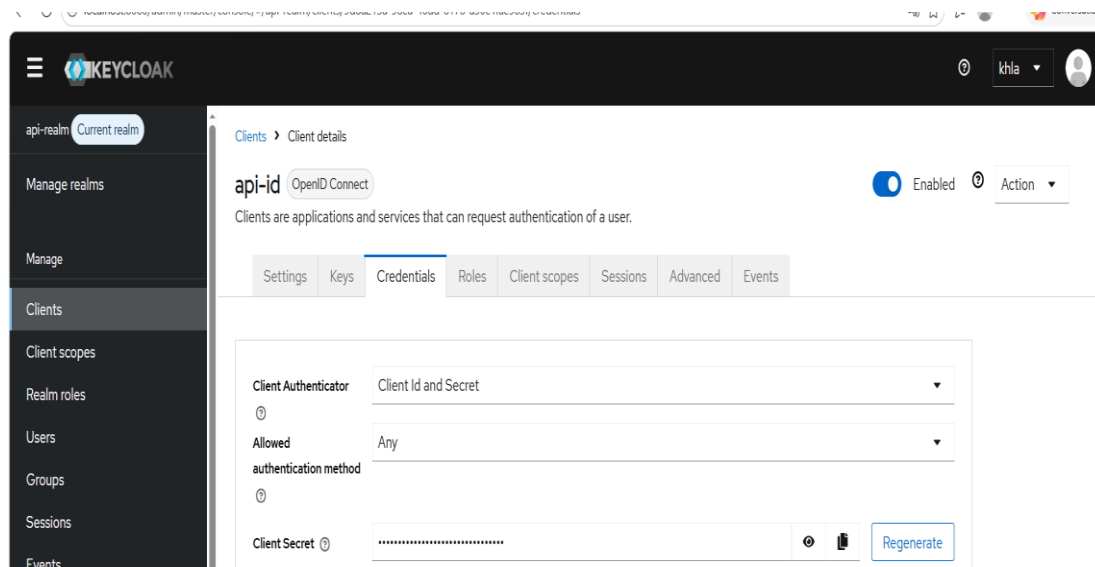


```
TP4-API JS database.js JS index.js {} keycloak-config.json X
> node_modules
JS database.js
JS index.js
{} keycloak-config.json
{} package-lock.json
{} package.json

{} keycloak-config.json > # confidential-port
1  {
2    "realm": "api-realm",
3    "auth-server-url": "http://localhost:8080",
4    "ssl-required": "external",
5    "resource": "api-id",
6    "credentials": {
7      "secret": "CkrfgKln8I7iBbMOMVyzE8P04it92Cot"
8    },
9    "confidential-port": 0
10 }
11
```

Le fichier keycloak-config.json contient l'URL du serveur d'authentification, le nom du realm, l'identifiant du client et son secret. Ces informations permettent à l'API de s'identifier auprès de Keycloak.

## Sécurisation des routes



Le middleware `keycloak.protect()` est ajouté sur les routes sensibles. Sans token valide dans le header `Authorization`, la requête est rejetée avec un statut 401 Unauthorized.

## Test — Obtention du token



[illegible]

## Test — Accès avec token

En incluant le token JWT dans le header Authorization (Bearer Token), la requête GET /personnes est autorisée et retourne les données. Sans token, la réponse est 401 Unauthorized.

## Flux de sécurisation

## Erreurs rencontrées et solutions

| Erreur                                      | Cause                                  | Solution                                    |
|---------------------------------------------|----------------------------------------|---------------------------------------------|
| Client not allowed for direct access grants | Direct Access Grants désactivé         | Activation dans Settings du client Keycloak |
| Invalid user credentials                    | Utilisateur créé dans le mauvais realm | Recréation dans api-realm                   |
| Account is not fully set up                 | Mot de passe temporaire activé         | Recréation avec Temporary = OFF             |

## Conclusion

L'intégration de Keycloak transforme notre API ouverte en API sécurisée par tokens JWT. Cette architecture respecte le principe de séparation des responsabilités : Keycloak gère l'identité, notre API gère la logique métier. OAuth 2.0 est le standard utilisé par Google, GitHub et toutes les grandes APIs modernes.

## Étape 7 (Optionnelle) : Documentation Swagger / OpenAPI

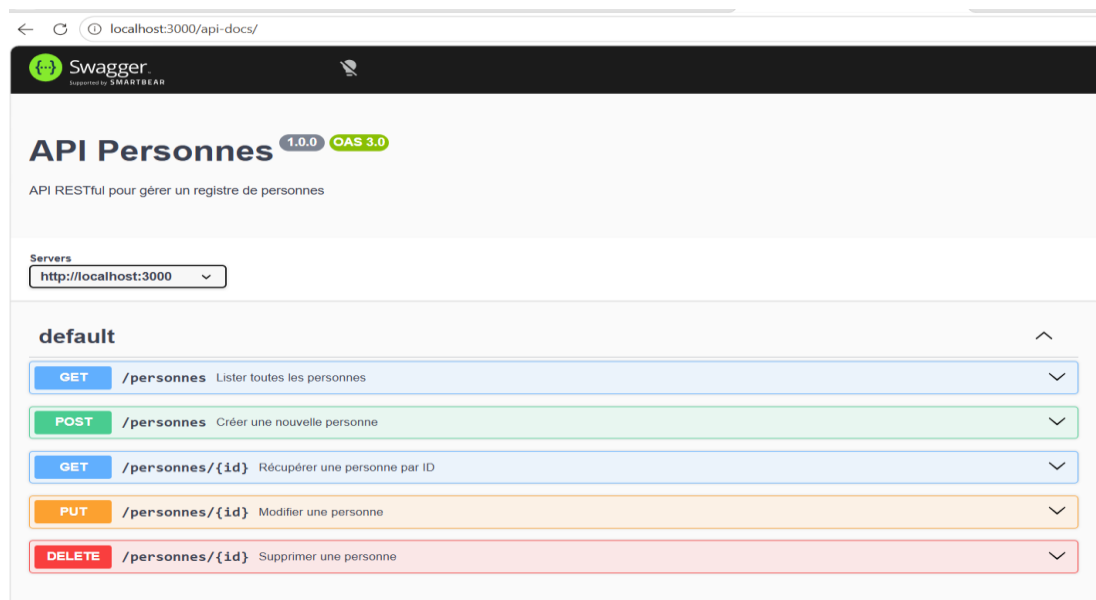
### Objectif

Générer une documentation interactive de l'API accessible depuis le navigateur via Swagger UI.

### Concepts théoriques

|                           |                                                                      |
|---------------------------|----------------------------------------------------------------------|
| <b>OpenAPI 3.0</b>        | Standard mondial pour décrire une API REST (format YAML ou JSON)     |
| <b>Swagger UI</b>         | Interface web auto-générée depuis la spec OpenAPI                    |
| <b>YAML</b>               | Format de configuration lisible basé sur l'indentation               |
| <b>swagger-ui-express</b> | Package qui génère l'interface web automatiquement                   |
| <b>yamljs</b>             | Package qui lit le fichier .yaml et le convertit en objet JavaScript |
| <b>/api-docs</b>          | Route d'accès à la documentation interactive                         |

### Interface Swagger UI



L'interface Swagger UI accessible sur <http://localhost:3000/api-docs> affiche toutes les routes de l'API avec leurs paramètres, corps de requête et codes de réponse. Le bouton "Try it out" permet de tester les routes directement depuis le navigateur.

### Commande d'installation

#### Terminal

```
npm install swagger-ui-express yamljs
```

### Conclusion

Swagger UI transforme l'API en documentation vivante et interactive. N'importe quel développeur peut comprendre toutes les routes, les paramètres attendus et les réponses possibles sans lire

une seule ligne de code. C'est une bonne pratique indispensable pour toute API destinée à être consommée par d'autres développeurs.

## Conclusion Générale

Ce TP nous a permis de concevoir et déployer une API RESTful complète en suivant les bonnes pratiques du développement moderne. Voici un bilan des compétences acquises :

| Compétence               | Acquis                                                     |
|--------------------------|------------------------------------------------------------|
| API REST avec Express.js | Création des 5 routes CRUD avec gestion des erreurs        |
| Base de données SQLite3  | Connexion, création de table, requêtes préparées           |
| Tests Postman            | Validation de toutes les routes et cas d'erreur            |
| OAuth 2.0 / Keycloak     | Sécurisation par tokens JWT, gestion des realms et clients |
| Documentation OpenAPI    | Spec YAML, Swagger UI, documentation interactive           |

Ce TP illustre l'architecture moderne d'une API de production : séparation des responsabilités (Express pour le routage, SQLite pour les données, Keycloak pour la sécurité, Swagger pour la documentation), utilisation des standards du secteur (REST, OAuth 2.0, OpenAPI 3.0) et validation par des tests systématiques.